

# 第一题：去括号 + 嵌套深度标注

## 文法 G

$S \rightarrow E \mid \varepsilon$

$E \rightarrow x \mid E + E \mid E - E \mid (E)$

## 示例

句子  $x$   $\rightarrow$  打印  $x0$

句子  $(x + ((x) + x) - (x - x))$

$\rightarrow$  打印  $x1 + x3 + x2 - x2 - x2$

## 翻译要求

- ① 去掉所有括号；
- ② 从左到右依次打印每个  $x$ ，并在其后加后缀 = 该  $x$  所在的括号嵌套深度；
- ③ 给出必要的符号说明与注释。

# 嵌套深度的属性

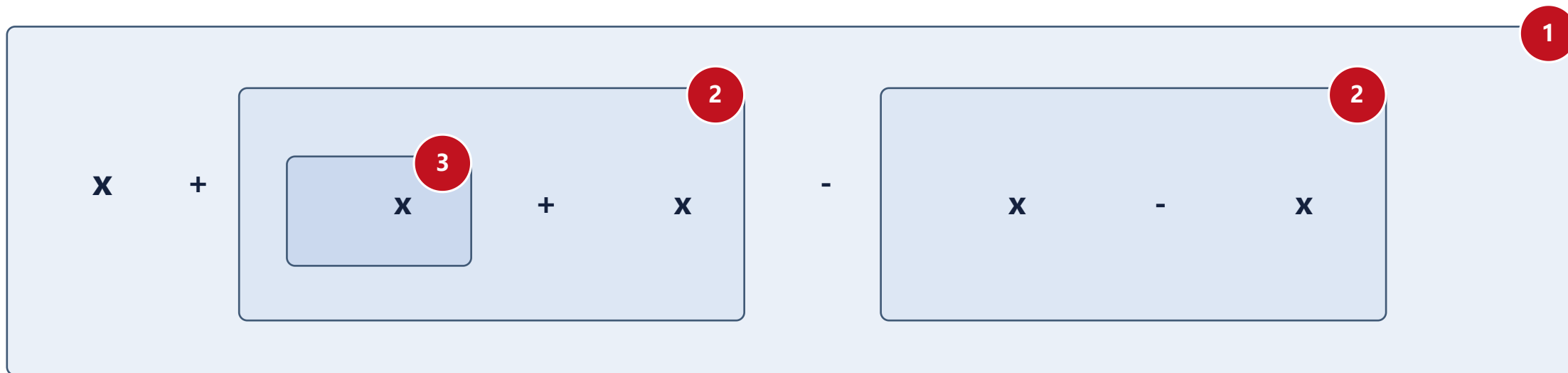
## 综合属性？继承属性？

一个  $x$  的深度，取决于它“上方”有几层括号。  
综合属性只能自下而上汇总，无法把“有几层括号”送下来。  
必须自上而下传递。

给  $E$  设继承属性  **$E.depth$** 。  
根处  $depth=0$ ；每遇 **(  $E$  )** 令内层  $depth+1$ ；  
运算符两侧子表达式  **$depth$  不变**；  
遇到  $x$  就打印 “ $x$ ” 和当前  $depth$ 。

# 按深度逐层标注

句子  $(x + ((x) + x) - (x - x))$  每对括号是一层；方框右上角的数字是该层的 depth。



打印结果（去括号，x 带深度后缀）



$x1 + x3 + x2 - x2 - x2$

# 语法制导定义 SDD

**符号说明：**  $E.depth$  ——  $E$  的继承属性，表示该子表达式所处的括号嵌套深度。 $print()$  为输出动作（副作用）。

| 产生式                       | 语义规则（属性计算 / 动作）                              | 说明                      |
|---------------------------|----------------------------------------------|-------------------------|
| $S \rightarrow E$         | $E.depth = 0$                                | 顶层表达式深度为 0              |
| $S \rightarrow \epsilon$  | —                                            | 空句子，无输出                 |
| $E \rightarrow x$         | $print( 'x' ); print( E.depth )$             | 打印 $x$ 及其当前深度后缀         |
| $E \rightarrow E_1 + E_2$ | $E_1 .depth = E.depth; E_2 .depth = E.depth$ | 运算符两侧深度不变，中间打印 $+$      |
| $E \rightarrow E_1 - E_2$ | $E_1 .depth = E.depth; E_2 .depth = E.depth$ | 同上，中间打印 $-$             |
| $E \rightarrow ( E_1 )$   | $E_1 .depth = E.depth + 1$                   | 进入一对括号，深度 $+1$ ；括号本身不打印 |

**性质：** 唯一的属性  $depth$  仅依赖父结点  $\rightarrow$  这是一个  $L$ -属性定义；打印为副作用，按从左到右遍历自然成序。

# 翻译方案 SDT：动作按位置嵌入

+语义动作 —— { } 的位置即执行时机：

算 depth 的动作放在子符号之前，打印运算符放在两子表达式之间。

$S \rightarrow \{ \text{E.depth} = 0 \} E$

$S \rightarrow \varepsilon$

$E \rightarrow x \{ \text{print('x')}; \text{print(E.depth)}; \}$

$E \rightarrow \{ E_1 \text{.depth} = E \text{.depth} \} E_1 + \{ \text{print('+')}; E_2 \text{.depth} = E \text{.depth} \} E_2$

$E \rightarrow \{ E_1 \text{.depth} = E \text{.depth} \} E_1 - \{ \text{print('-')}; E_2 \text{.depth} = E \text{.depth} \} E_2$

$E \rightarrow ( \{ E_1 \text{.depth} = E \text{.depth} + 1 \} E_1 )$

**动作** 在进入子表达式前设置其继承属性 depth;

**动作** 在子树处理完后打印 (x 处打 x+深度, 运算符处打 + / -) 。

括号产生式只调整 depth、不输出括号。

## 第二题：声明 + 类型检查 的翻译方案

### 文法 (声明部分 D + 可执行部分 S)

$P \rightarrow D S$

$D \rightarrow T \text{ idlist} ; D \mid L ; D \mid \epsilon$

$T \rightarrow \text{int} \mid \text{float} \mid \text{char}$

$L \rightarrow \text{Array id} [ \text{numlist} ]$

$\mid L , \text{id} [ \text{numlist} ]$

$\text{idlist} \rightarrow \text{id} \mid \text{id} , \text{idlist}$

$\text{numlist} \rightarrow \text{num} \mid \text{num} , \text{numlist}$

$S \rightarrow \text{id} = E ;$

$\mid \text{id} [ \text{numlist} ] = E ;$

$\mid S ; S$

$E \rightarrow E + E \mid \text{id} \mid \text{id} [ \text{numlist} ] \mid \text{num}$

### 要求 (1~4)

- 程序 = 声明部分 + 可执行部分；
- 类型由 T 声明、数组形状由 Array(L) 声明，可分处不同语句；
- int/float 各 4 字节，char 1 字节；每个变量存储须为 4 的整数倍；
- 可执行部分只有赋值语句与表达式。

**示例：** int a, b; Array a[10,20],  
c[10,20]; float c;  $\Rightarrow$  a 是二维 int 数  
组，c 是二维 float 数组，b 是 int 标量。

# 要求

## ①重复声明检查

同名变量被重复声明时报错，须指出：第几条声明语句、哪个变量、重复的是基本类型还是数组形状（语句号从 1 起）；

②**数组统计与排序**：统计共声明几个数组，并按所占存储空间从大到小输出数组名与字节数；char 数组按“4 字节整数倍”向上取整；

③**类型 + 形状检查**：可执行部分变量须先声明后使用；a.运算/赋值两侧基础类型一致；b.数组引用的维数与下标范围须与声明形状相容；

④**先赋值后引用**：可执行部分中的变量必须先被赋初值、再被引用，否则报错（须在方案的语义动作中体现该检查）。

# 符号表，声明阶段建表、执行阶段查表

## 符号表项设计

| 名称                         | 作用                                                      |
|----------------------------|---------------------------------------------------------|
| name                       | 变量名（关键字）                                                |
| hasType/type               | 是否已声明基本类型；<br>int/float/char                            |
| hasShape/dims              | 是否已声明数组形状；各维维展<br>[d <sub>1</sub> ,...,d <sub>k</sub> ] |
| typeStmtNo/shapeStmt<br>No | 记录声明所在语句号，报错定位                                          |
| inited                     | (执行阶段) 是否已赋初值                                           |

### 声明阶段 P → D ...

顺序遍历 D：维护语句号计数器，对每个 id 调用建表/查重 → 完成要求 a；记录类型与形状。

### 数组汇总 D 结束时

扫描符号表中 hasShape 的项，算存储大小、按降序排序输出 → 完成要求 b。

### 执行阶段 S / E

查表做 ‘先声明后使用’ 与类型/形状检查 (c)、‘先赋值后引用’ 检查 (d) ； E 综合出 E.type。



## a)重复声明检查 (区分基本类型 / 数组形状)

**关键：**同名出现在“一条 T 声明 + 一条 Array 声明”中是合法的（声明数组的合法方式）。  
只有 **两次基本类型声明** 或 **两次数组形状声明** 才算重复。故用 hasType 与 hasShape 两个标志分别判断

```
// stmtNo: 全局语句号计数器, 每进入一条 T/L 声明语句前 +1  
D → T idlist ; D { 对 idlist 中每个 id 调用 declType(id, T.type, stmtNo) }  
D → L ; D { L 的语义动作对每个 id 调用 declShape(id, dims, stmtNo) }
```

```
declType(id, t, k): e=lookup(id)||enter(id);  
    if e.hasType → 报错(“语句 k: 变量 id 基本类型重复声明”);  
    else e.hasType=true; e.type=t; e.typeStmtNo=k;  
declShape(id, dims, k): e=lookup(id)||enter(id);  
    if e.hasShape → 报错(“语句 k: 变量 id 数组形状重复声明”);  
    else e.hasShape=true; e.dims=dims; e.shapeStmtNo=k;
```

**报错信息：**①语句号 k (从 1 起)、②变量名 id、③重复类别 (基本类型 / 数组形状)

## b)数组统计与按存储大小降序输出

### 存储大小怎么算?

元素数 = 各维维展之积  $d_1 \times \dots \times d_k$

元素字节: int/float=4, char=1

原始字节 = 元素数  $\times$  元素字节

向上取整到 4 的整数倍 (char 数组关键) :

**size =  $\lceil \text{raw} / 4 \rceil \times 4$**

### arraySize(e):

```
n = product(e.dims)
raw = n * elemSize(e.type)
return ((raw + 3) / 4) * 4
```

### reportArrays(): // 在 D 归约完成后调用

```
A = { e ∈ 符号表 | e.hasShape }
打印 |A| (数组个数)
按 arraySize 降序打印 (name, size)
```

示例 int a,b; Array a[10,20],c[10,20]; float c;

| 数组 | 基本类型  | 维展      | 元素数 | 元素字节 | 存储大小   |
|----|-------|---------|-----|------|--------|
| a  | int   | 10 × 20 | 200 | 4    | 800 字节 |
| c  | float | 10 × 20 | 200 | 4    | 800 字节 |

共 2 个数组;  
大小相同则可按名字或声明确定report的顺序。  
若为 char 数组 [10,20]: raw=200 → 已是 4 倍; 若 [3,3]=9 → 取整为 12。

## c) 先声明后使用 + 基础类型 / 数组形状检查

// 表达式综合出基础类型 E.type; 冲突即报错

$E \rightarrow \text{num} \quad \{ E.\text{type} = \text{int} \}$

$E \rightarrow \text{id} \quad \{ \text{checkDeclared}(\text{id}); \text{checkInit}(\text{id});$   
 $\quad \quad \quad E.\text{type} = \text{id.type} \}$

$E \rightarrow \text{id} [\text{numlist}]$   
 $\{ \text{checkDeclared}(\text{id}); \text{checkShape}(\text{id}, \text{numlist});$   
 $\quad \quad \quad E.\text{type} = \text{id.type} \}$

$E \rightarrow E_1 + E_2$   
 $\{ \text{if } E_1.\text{type} \neq E_2.\text{type} \rightarrow \text{报错(类型不一致)};$   
 $\quad \quad \quad E.\text{type} = E_1.\text{type} \}$

$S \rightarrow \text{id} = E; \{ \text{if } \text{id.type} \neq E.\text{type} \rightarrow \text{报错} \}$

**checkShape(id, subs)**

① **维数一致**: 下标个数 = id.dims 的维数;

② **范围合法**:

每个下标 num 满足  $0 \leq \text{num} < \text{对应维展}$ 。

以 a 声明为 a[10,20] 为例:

a[5,17] ✓ (2 维,  $5 < 10$ 、 $17 < 20$ )

a[8] ✗ 维数不符 ( $1 \neq 2$ )

a[16,18] ✗  $16 \geq 10$  越界

**checkDeclared:**

查符号表, 未声明即报“未声明”;

类型一致性对 + 与 = 两侧均要求 **基础类型相同** (int/float/char 不可混用)

## d) 先赋初值、后引用（在动作中体现）

**做法：**每个符号表项加 inited 标志。引用（读）一个变量前先 checkInited；赋值语句要 **先处理右部 E（其中的引用都要已初始化）**，再把左部标记为已初始化——顺序很关键。

**checkInited(id):** if not id.inited → 报错(“变量 id 在赋初值前被引用”)

$E \rightarrow id \quad \{ \text{checkDeclared}(id); \text{checkInited}(id); E.type = id.type \}$

$S \rightarrow id = E; \{ /*\text{先算右部}*/ \text{checkType}(id, E); \text{id.inited} = \text{true} /*\text{后置初始化}*/ \}$

$S \rightarrow id [ \text{numlist} ] = E;$

$\{ \text{checkShape}(id, \text{numlist}); \text{checkType}; \text{id.inited} = \text{true} \}$

对  $id = E$ ，必须在归约动作里 ‘先检查 E 中各引用、最后才置  $id.inited = \text{true}$ ’；否则形如  $a = a + 1$  在 a 尚未初始化时会被漏报。S ; S 顺序执行使 inited 状态沿语句流向后传播。

## 整合 a-d: 声明部分

```
P → D { reportArrays() } S
D → T { stmtNo++; idlist.bt=T.type; idlist.k=stmtNo } idlist ; D
D → { stmtNo++; L.k=stmtNo } L ; D      D → ε
T → int | float | char { T.type 取相应值 }
idlist → id { declType(id, idlist.bt, idlist.k) }
idlist → id , { declType(id,...); idlist1 .bt=idlist.bt; idlist1 .k=idlist.k } idlist1
L → Array id [ numlist ] { declShape(id, numlist.dims, L.k) }
L → { L1 .k=L.k } L1 , id [ numlist ] { declShape(id, numlist.dims, L.k) }
numlist → num { dims=[num.val] }
numlist → num , numlist1 { dims = num.val : numlist1 .dims }
```

**声明阶段:** 顺序遍历 D, stmtNo 给声明语句编号; declType / declShape 借 hasType / hasShape 查重 (要求 a) ; num.val 是 num 的字面值。

## 整合 a-d: 可执行部分

$S \rightarrow id = E ; \{ \text{checkDeclared}(id); \text{若 } \text{typeOf}(id) \neq E.type \text{ 报错}; \text{markInited}(id) \}$

$S \rightarrow id [ \text{numlist} ] = E ;$

$\{ \text{checkDeclared}(id); \text{checkShape}(id, \text{numlist.dims});$

$\text{若 } \text{typeOf}(id) \neq E.type \text{ 报错}; \text{markInitedElem}(id, \text{numlist.dims}) \}$

$S \rightarrow S_1 ; S_2$  (从左到右归约, 初始化状态自然向后传播)

$E \rightarrow \text{num} \{ E.type = \text{int} \}$

$E \rightarrow id \{ \text{checkDeclared}(id); \text{checkInited}(id); E.type = \text{typeOf}(id) \}$

$E \rightarrow id [ \text{numlist} ]$

$\{ \text{checkDeclared}(id); \text{checkShape}(id, \text{numlist.dims});$

$\text{checkInitedElem}(id, \text{numlist.dims}); E.type = \text{typeOf}(id) \}$

$E \rightarrow E_1 + E_2 \{ \text{若 } E_1.type \neq E_2.type \text{ 报错}; E.type = E_1.type \}$

**动作 ↔ 要求:** a = declType / declShape; b = reportArrays; c = checkDeclared + 类型一致 + checkShape; d = checkInited (标量) / checkInitedElem (数组, 常量下标元组)。

动作在产生式末尾 → 右部 E 先归约, 故对 E 中各引用的检查先于对左部的 markInited (读取检查在前、写入标记在后)。

## f.为什么数组形状检查能在编译期完成?

“形状”与“下标”都是编译期常量

声明侧 `Array id [ numlist ]` 与引用侧 `id [ numlist ]` 的下标都由 **numlist**  $\rightarrow$  **num | num** , **numlist** 产生, 全部是字面常量 num —— 编译时即可知道维数与每个下标的具体值。

- 下标可为变量 / 表达式?
- 维数仍可静态检查 (结构信息不依赖运行期值) ;
- 但下标范围一般要到运行期才知道, 需插入运行期越界检查 ...

## g.能否检查任意数组引用的“先赋值后引用”?

用“每个变量一个 `inited` 标志”的方案, 对数组不行。

数组是按元素赋值的: `a[5,17]=...` 只初始化了一个元素, 单个布尔标志无法表达“哪些元素已初始化”。把整个数组当作已初始化会漏报对未初始化元素的读取...

每个 `a[c1 , ..., ck ]` 指向固定元素。  
改为按“(数组, 常量下标元组)”集合跟踪初始化, 是可以检查的。



## 第八次作业

截止日期: 2026.05.07

- 练习5.1.1: 考虑文法

$$S \rightarrow E \ n$$
$$E \rightarrow E - T \mid T$$
$$T \rightarrow T / F \mid F$$
$$F \rightarrow ( E ) \mid \text{digit}$$

其中  $S, E, T, F$  为非终结符

1. 消除左递归
2. 对消除左递归后的文法, 给出一个语法制导定义, 使得  $S.val$  为表达式  $S$  的值。注:  $\text{digit.lexval}$  表示数字字面量的值
3. 使用上面得到的 SDD, 给出  $8 / 4 - 3 \ n$  的注释语法分析树





# 第八次作业

截止日期: 2026.05.07

- 练习5.1.1: 考虑文法

$$S \rightarrow E n$$

$$E \rightarrow E - T \mid T$$

$$T \rightarrow T / F \mid F$$

$$F \rightarrow ( E ) \mid \text{digit}$$

其中 S, E, T, F 为非终结符

1. 消除左递归

E、T 都是直接左递归, 分别套用公式:

$$E: \alpha = "- T", \beta = T$$

$$T: \alpha = "/ F", \beta = F$$

$$A \rightarrow A \alpha \mid \beta \Rightarrow A \rightarrow \beta A', A' \rightarrow \alpha A' \mid \varepsilon$$

把“左递归的循环”改写成“右侧的尾递归 +  $\varepsilon$  收尾”,  
 $\beta$  提到最前面。

消除左递归后的文法

$$S \rightarrow E n$$

$$E \rightarrow T E'$$

$$E' \rightarrow - T E' \mid \varepsilon$$

$$T \rightarrow F T'$$

$$T' \rightarrow / F T' \mid \varepsilon$$

$$F \rightarrow ( E ) \mid \text{digit}$$



## 第八次作业

截止日期: 2026.05.07

- 练习5.1.1: 考虑文法

$$S \rightarrow E n$$
$$E \rightarrow E - T \mid T$$
$$T \rightarrow T / F \mid F$$
$$F \rightarrow ( E ) \mid \text{digit}$$

其中  $S, E, T, F$  为非终结符

1. 消除左递归
2. 对消除左递归后的文法, 给出一个语法制导定义, 使得  $S.val$  为表达式  $S$  的值。注:  $\text{digit.lexval}$  表示数字字面量的值

消完左递归后文法变成了右递归, 运算符和它的两个操作数被拆到了不同产生式里—— $E' \rightarrow - T E'$  这条规则里, 减号左边那个操作数不在它的子树里, 而在它"左边"。

用一个继承属性把"左边已经算好的部分结果"传进来



## 第八次作业

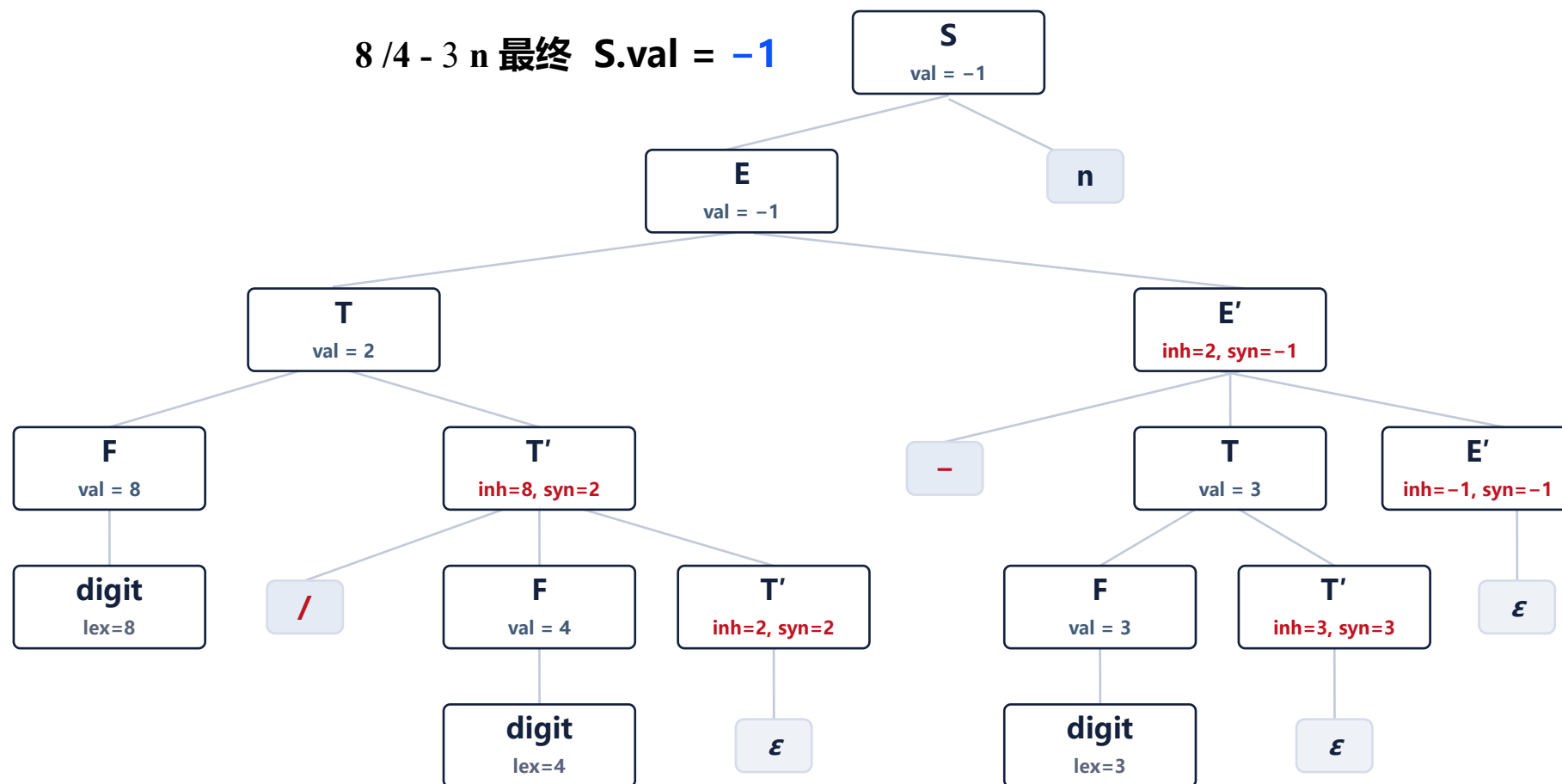
截止日期: 2026.05.07

**属性约定:**  $E'$ 、 $T'$  用继承属性 inh (已累积的左部值) 与综合属性 syn (子树最终值) ;  
其余非终结符用综合属性 val。

| 产生式                          | 语义规则                                           |
|------------------------------|------------------------------------------------|
| $S \rightarrow E n$          | $S.val = E.val$                                |
| $E \rightarrow T E'$         | $E'.inh = T.val; E.val = E'.syn$               |
| $E' \rightarrow - T E'_1$    | $E'_1.inh = E'.inh - T.val; E'.syn = E'_1.syn$ |
| $E' \rightarrow \epsilon$    | $E'.syn = E'.inh$                              |
| $T \rightarrow F T'$         | $T'.inh = F.val; T.val = T'.syn$               |
| $T' \rightarrow / F T'_1$    | $T'_1.inh = T'.inh / F.val; T'.syn = T'_1.syn$ |
| $T' \rightarrow \epsilon$    | $T'.syn = T'.inh$                              |
| $F \rightarrow ( E )$        | $F.val = E.val$                                |
| $F \rightarrow \text{digit}$ | $F.val = \text{digit.lexval}$                  |

# 第八次作业

截止日期: 2026.05.07



## 第八次作业

截止日期: 2026.05.07

- 练习5.1.2: 考虑产生式  $A \rightarrow BCD$ , 其中  $A$ 、 $B$ 、 $C$ 、 $D$  四个非终结符各有综合属性  $s$  和继承属性  $i$ 。对于下面的规则
  - a)  $A.s = B.s + C.s + D.s$
  - b)  $B.i = A.i$ ;  $C.i = B.s$ ;  $D.i = B.s + C.s$ ;  $A.s = B.s$ ;
  - c)  $B.i = C.s$ ;  $C.i = B.s + 1$ ;  $A.s = B.s$

分别讨论

① 是 S-属性吗?

只设置了头部的综合属性、且只用右部符号的综合属性。

② 是 L-属性吗?

逐条看每个继承属性  $X_j.i$ : 是否只依赖头部  $A$  的继承属性、或它左边  $X_1 \dots X_{j-1}$  的属性。出现“右兄弟依赖”即不是。

③ 有一遍求值顺序吗?

画依赖图找环。注意补上隐含边: 一个符号的综合属性通常依赖它自己的继承属性 ( $X.s$  依赖  $X.i$ )。无环  $\Rightarrow$  有顺序。

# 第八次作业

截止日期: 2026.05.07

- 练习5.1.2: 考虑产生式  $A \rightarrow BCD$ , 其中 A、B、C、D 四个非终结符各有综合属性 s 和继承属性 i。对于下面的规则
  - $A.s = B.s + C.s + D.s$
  - $B.i = A.i; C.i = B.s; D.i = B.s + C.s; A.s = B.s;$
  - $B.i = C.s; C.i = B.s + 1; A.s = B.s$

分别讨论

(a)  $A.s = B.s + C.s + D.s$

S-属性? **是**。只设置了 A 的综合属性,

L-属性?

**是**。S-属性是 L-属性的特例 (没给右部设继承属性)。

求值顺序?

**存在**。自底向上先算 B.s、C.s、D.s, 再算 A.s

(b)  $B.i = A.i; C.i = B.s; D.i = B.s + C.s; A.s = B.s$

S-属性? **不是** (给 B、C、D 设了继承属性)。

L-属性?

**是**, 逐条都只看左边:

- $B.i = A.i$ : 用 A 的继承属性 ✓
- $C.i = B.s$ : B 在 C 左边 ✓
- $D.i = B.s + C.s$ : B、C 都在 D 左边 ✓

求值顺序?

**存在** (从左到右一遍, 递归下降即可):

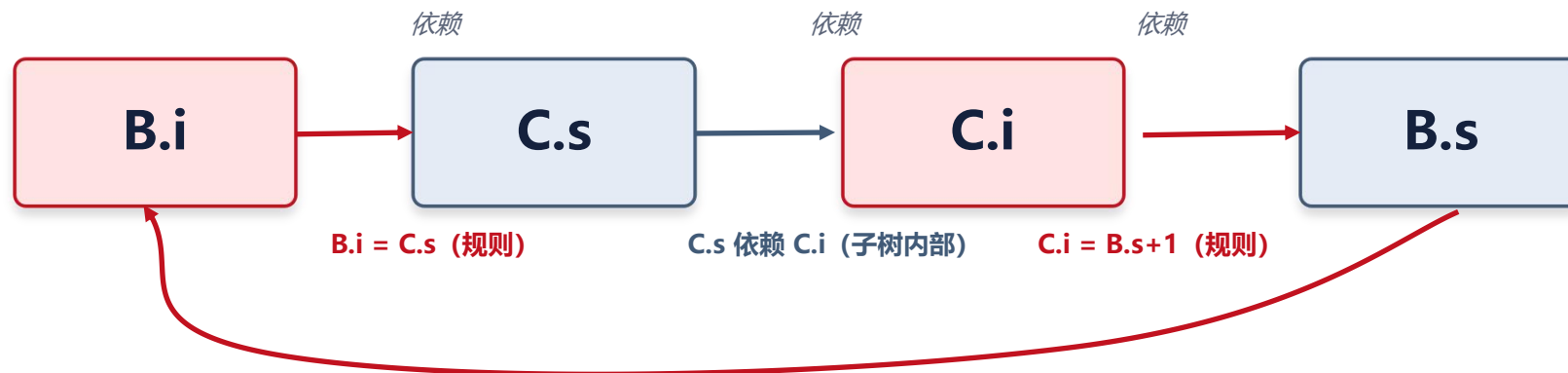
$B.i = A.i \rightarrow \text{算} B \rightarrow C.i = B.s \rightarrow \text{算} C \rightarrow D.i = B.s + C.s \rightarrow \text{算} D \rightarrow A.s = B.s$

# 第八次作业

截止日期: 2026.05.07

- 练习5.1.2: 考虑产生式  $A \rightarrow BCD$ , 其中  $A$ 、 $B$ 、 $C$ 、 $D$  四个非终结符各有综合属性  $s$  和继承属性  $i$ 。对于下面的规则
  - a)  $A.s = B.s + C.s + D.s$
  - b)  $B.i = A.i$ ;  $C.i = B.s$ ;  $D.i = B.s + C.s$ ;  $A.s = B.s$ ;
  - c)  $B.i = C.s$ ;  $C.i = B.s + 1$ ;  $A.s = B.s$

分别讨论





## 第九次作业

截止日期: 2026.05.18

- 练习6.1.1: 为下列表达式构建 DAG 并指出每个子表达式的值编码

$$((\underline{a + b}) * (\underline{a} + (\underline{a + b}))) - ((a + b) * (a - b))$$

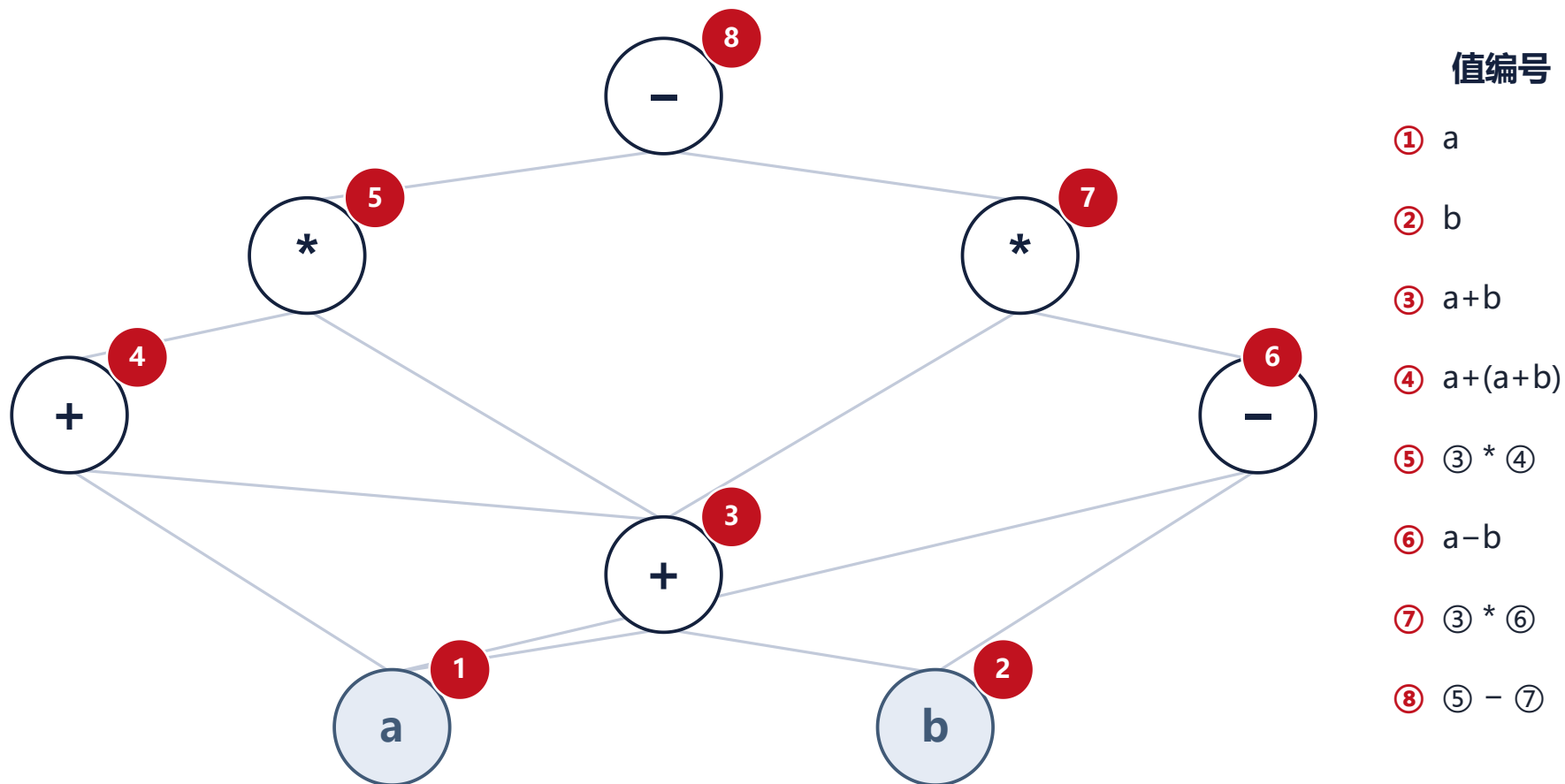
自底向上、给每个子表达式一个**值编号 (value number)** ;  
遇到 “运算符 + 相同操作数编号” 的组合时不新建结点, 直接复用已有编号



# 第九次作业

截止日期: 2026.05.18

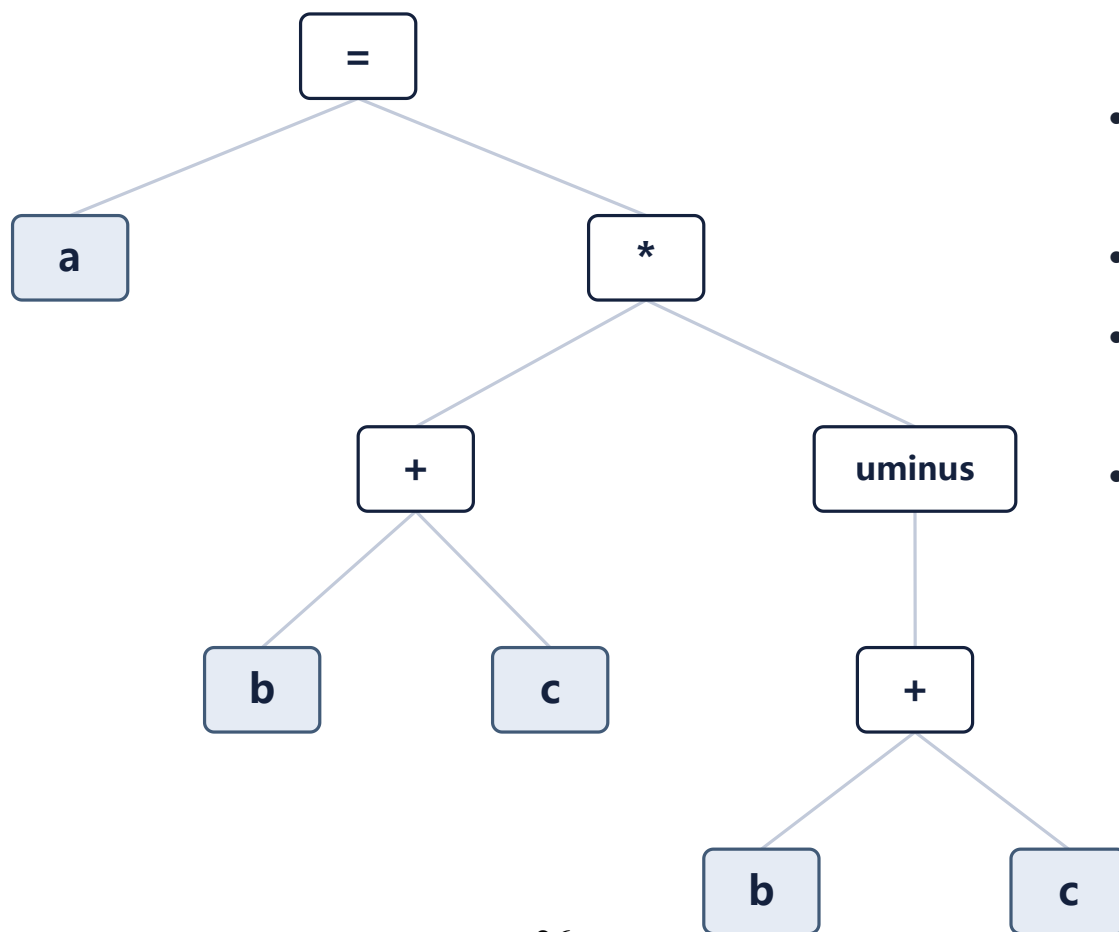
$$((a + b) * (a + (a + b))) - ((a + b) * (a - b))$$



## 第九次作业

截止日期: 2026.05.18

- 练习6.1.2: 将语句  $a = (b + c) * (- (b + c))$  翻译为



- = 是赋值结点, 左是 a、右是表达式;
- uminus (一元负号) 只有一个孩子;
- $(b+c)$  在语法树里出现两次  $\rightarrow$  两棵独立子树;
- 若改用 DAG, 两个  $(b+c)$  会合并为一个结点 (与 6.1.1 同理)。



## 第九次作业

截止日期: 2026.05.18

- 练习6.1.2: 将语句  $a = (b + c) * (- (b + c))$  翻译为

(b) 四元式 Quadruples

| # | op     | arg1 | arg2 | result |
|---|--------|------|------|--------|
| 0 | +      | b    | c    | t1     |
| 1 | +      | b    | c    | t2     |
| 2 | uminus | t2   |      | t3     |
| 3 | *      | t1   | t3   | t4     |
| 4 | =      | t4   |      | a      |

(c) 三元式 Triples

| # | op     | arg1 | arg2 |
|---|--------|------|------|
| 0 | +      | b    | c    |
| 1 | +      | b    | c    |
| 2 | uminus | (1)  |      |
| 3 | *      | (0)  | (2)  |
| 4 | =      | a    | (3)  |

(d) 间接三元式

指令表 (指针)

| 序  | 指向  |
|----|-----|
| 35 | (0) |
| 36 | (1) |
| 37 | (2) |
| 38 | (3) |
| 39 | (4) |



## 第九次作业

- 练习6.1.3：确定下列声明序列中各个标识符的类型和相对地址。

```
float x;  
record { float x; float y; } p;  
record {  
    record { int tag; float x; } m;  
    record { int idx; float y; } n;  
} q;
```

### 计算规则与假设

- 宽度假设：int = 4, float = 8, 不考虑对齐填充;
- 顶层声明：偏移从 0 起, 依次累加各自宽度;
- 记录类型：其宽度 = 各字段宽度之和;
- 进入 record 时压入新作用域, 字段偏移从 0 重新计数 (相对该记录起点)。

注:采用int=4 float=4只要算对了就可以



## 第九次作业

- 练习6.1.3：确定下列声明序列中各个标识符的类型和相对地址。

| 标识符     | 类型                             | 相对地址 | 宽度 |
|---------|--------------------------------|------|----|
| x       | float                          | 0    | 8  |
| p       | record { float x; float y; }   | 8    | 16 |
| p.x     | float                          | 0    | 8  |
| p.y     | float                          | 8    | 8  |
| q       | record { m:record; n:record; } | 24   | 24 |
| q.m     | record { int tag; float x; }   | 0    | 12 |
| q.m.tag | int                            | 0    | 4  |
| q.m.x   | float                          | 4    | 8  |
| q.n     | record { int idx; float y; }   | 12   | 12 |
| q.n.idx | int                            | 0    | 4  |
| q.n.y   | float                          | 4    | 8  |

相对地址都相对“最近的外层记录 / 顶层数据区”起点。

顶层：x@0、p@8、q@24（总占48字节）；记录内字段各自从0起算（如q.m.tag@0、q.m.x@4）。



## 第九次作业

练习6.1.4: 考虑龙书图 6-22 的翻译方案, 翻译下列赋值语句

$x = a[b[i][j]][c[k]];$

并给出注释语法分析树。

$L \rightarrow id [ E ]$ : 记录数组 base 与元素类型,  
offset = E.addr  $\times$  元素宽度;  
 $L \rightarrow L1 [ E ]$ :  
offset = L1.offset + E.addr  $\times$  元素宽度 (逐维累加);  
 $E \rightarrow L$ :  
t = base[offset] (真正取值)。

翻译  $x = a[b[i][j]][c[k]]$  (用符号宽度 w)

// 先算内层下标 b[i][j]

$t1 = i * w$

$t2 = j * w ; t3 = t1 + t2$

$t4 = b[t3]$

// 内层下标 c[k]

$t5 = k * w ; t6 = c[t5]$

// 外层 a[t4][t6]

$t7 = t4 * w$

$t8 = t6 * w ; t9 = t7 + t8$

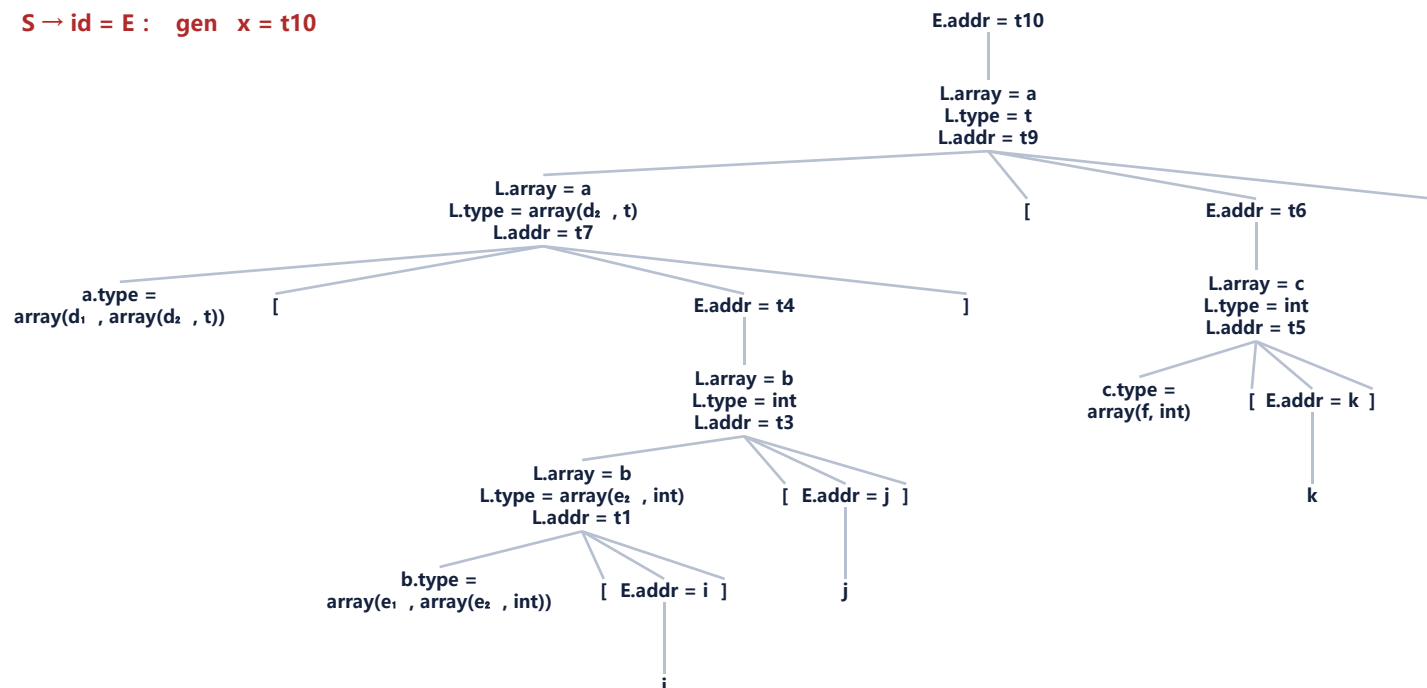
$t10 = a[t9]$

$x = t10$

# 第九次作业

## $x = a[b[i][j]][c[k]]$ 的注释语法分析树

$S \rightarrow id = E : \text{gen } x = t10$



**记法:** E.addr=子表达式的值/地址 · L.array 数组名 · L.type 当前残余类型 · L.addr 累计字节偏移 · id 叶标注其类型

**类型设:** a:array(d<sub>1</sub>, array(d<sub>2</sub>, t)) · b:array(e<sub>1</sub>, array(e<sub>2</sub>, int)) · c:array(f, int) (维数未给, 用符号; 各层宽度取该层 L.type 的宽度)

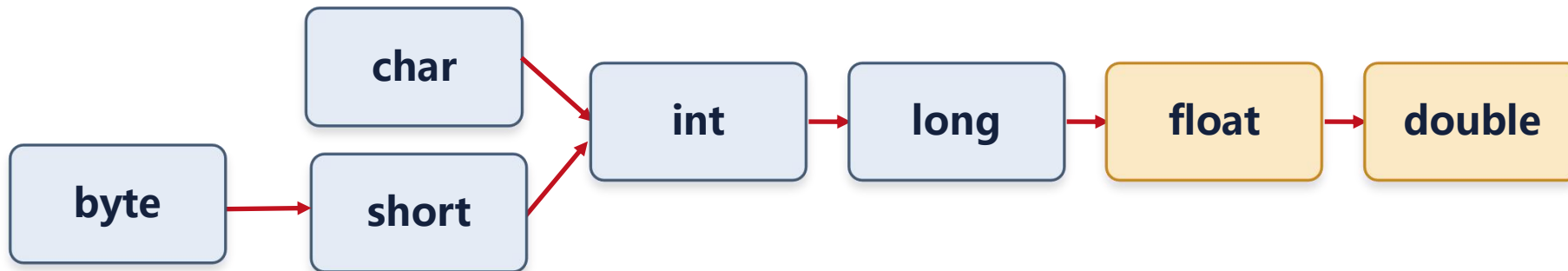
**对应:** 临时量 t1...t10 与第 9 页三地址码一致 (即教材 图 6-24)。

# 第十次作业

截止日期: 2026.05.23

- 练习6.2.1: 假定图 6-26 中的函数 `widen` 可以处理图 6-25a 的层次结构中的所有类型, 翻译下列表达式。假定 `c` 是 `char` 型, `i` 是 `int` 型, `s` 是 `short` 型, `f` 是 `float` 型, `y` 是 `double` 型。

低类型可安全转成高类型:



`widen(a, t, w)`

把类型为 `t` 的值 `a` 加宽成类型 `w`:

若 `t=w` 不动;

否则生成一个临时变量与一条类型转换指令 `(w) a`;

沿层次逐级向上转换。

## 二元运算的处理

对 `E1 op E2`:

先取 `m = max(E1.type, E2.type)` (层次里较高者);

把两个操作数各自 `widen` 到 `m`, 再做 `m` 类型的运算;

结果类型为 `m`。

赋值 `lhs = rhs` 时再把 `rhs` 加宽到 `lhs` 的类型。



## 第十次作业

截止日期: 2026.05.23

- 练习6.2.1: 假定图 6-26 中的函数 `widen` 可以处理图 6-25a 的层次结构中的所有类型, 翻译下列表达式。假定 `c` 是 `char` 型, `i` 是 `int` 型, `s` 是 `short` 型, `f` 是 `float` 型, `y` 是 `double` 型。

### ① `f = i * c`

```
t1 = (int) c    // char → int
t2 = i * t1     // int 乘法
t3 = (float) t2 // int → float
f = t3
```

*max(int, char)=int: 先把 `c` 加宽到 `int` 做乘法;  
结果 `int` 再加宽到 `float` 存入 `f`。*

### ② `y = (i + f) - (s + c)`

```
u1 = (float) i // i+f: i→float
u2 = u1 + f    // float 加法
u3 = (short) c // s+c: c→short
u4 = s + u3    // short 加法
u5 = (float) u4 // short→float
u6 = u2 - u5    // float 减法
u7 = (double) u6 // float→double
y = u7
```

*`i+f` 在 `float`, `s+c` 在 `short`;  
两者相减取 `max=float`;  
最后整体加宽到 `double` 存入 `y`。*



## 第十次作业

截止日期：2026.05.23

练习6.2.2：在图 6-36 的语法制导定义中添加处理下列控制流构造的规则：

1.  $S \rightarrow \text{repeat } S_1 \text{ until } B$ ，当  $B$  为真时结束循环
2.  $S \rightarrow \text{for } ( S_1 ; B ; S_2 ) S_3$

### **S.next (继承)**

S 执行完后应跳转到的标号——即 S 之后那条指令的位置。

### **B.true / B.false (继承)**

布尔表达式 B 为真 / 为假时分别跳转到的标号。

### **newlabel()**

生成一个全新的、尚未使用的标号名。

### **label(L)**

把标号 L 附到下一条生成的指令上，即生成一行 “L:”。

循环就是 “在合适的位置放标号，再让 B 的真假分支跳到正确的标号

## 第十次作业

截止日期: 2026.05.23

练习6.2.2: 在图 6-36 的语法制导定义中添加处理下列控制流构造的规则:

1.  $S \rightarrow \text{repeat } S_1 \text{ until } B$ , 当  $B$  为真时结束循环

```
begin = newlabel()
S1.next = newlabel()
```

```
B.true = S.next  // B 真 → 退出循环
B.false = begin  // B 假 → 回到开头
```

```
S.code = label(begin)
    || S1.code
    || label(S1.next)
    || B.code
```

```
begin:
    (S1 的代码)
```

```
S1.next:
    (B 的代码)
    B 真 → 跳 S.next
    B 假 → 跳 begin
(S.next): ...
```

**repeat 是“先执行后判断”——所以  $S_1$  在前、 $B$  在后; 与 while 相反, 是  $B$  为假时回跳 begin、 $B$  为真时退出 ( $B.\text{true}=S.\text{next}$ )。循环体至少执行一次。**

## 第十次作业

截止日期: 2026.05.23

练习6.2.2: 在图 6-36 的语法制导定义中添加处理下列控制流构造的规则:

1.  $S \rightarrow \text{for} ( S_1 ; B ; S_2 ) S_3$

**S1.next** = newlabel() // 测试 B 的位置

**B.true** = newlabel() // 循环体 S3

**B.false** = S.next

**S3.next** = newlabel() // 更新语句 S2

**S2.next** = S1.next

**S.code** = S1.code

|| label(S1.next) || B.code

|| label(B.true) || S3.code

|| label(S3.next) || S2.code

|| gen('goto' S1.next)

(S1 初始化)

**S1.next:**

(B 的代码) 假  $\rightarrow$  S.next

**B.true:**

(S3 循环体)

**S3.next:**

(S2 更新)

goto S1.next

(S.next): ...

## 第十次作业

- 练习6.2.3：使用图 6-43 中的翻译方案翻译下列表达式。给出每个子表达式的 **truelist** 和 **falselist**。你可以假设第一条被生成的指令的地址是 100。

1.  $a==b \ \&\& \ (c==d \ || \ e==f)$

```

100: if a==b goto 102
101: goto _ ← 整体 F
102: if c==d goto _ ← 整体 T
103: goto 104
104: if e==f goto _ ← 整体 T
105: goto _ ← 整体 F
    
```

| 子表达式               | truelist         | falselist        |
|--------------------|------------------|------------------|
| $a==b$             | {100}            | {101}            |
| $c==d$             | {102}            | {103}            |
| $e==f$             | {104}            | {105}            |
| $c==d \    \ e==f$ | {102,104}        | {105}            |
| <b>整体</b>          | <b>{102,104}</b> | <b>{101,105}</b> |

$\&\&$  回填左真表:  $backpatch(\{100\}, 102) \rightarrow 100$  跳 102;  $\ || \$  回填左假表:  $backpatch(\{103\}, 104) \rightarrow 103$  跳 104。

## 第十次作业

- 练习6.2.3：使用图 6-43 中的翻译方案翻译下列表达式。给出每个子表达式的 **truelist** 和 **falselist**。你可以假设第一条被生成的指令的地址是 100。
  - $(a==b \parallel c==d) \parallel e==f$

```

100: if a==b goto __ ← 整体 T
101: goto 102
102: if c==d goto __ ← 整体 T
103: goto 104
104: if e==f goto __ ← 整体 T
105: goto __ ← 整体 F
    
```

| 子表达式                  | truelist             | falselist    |
|-----------------------|----------------------|--------------|
| $a==b$                | {100}                | {101}        |
| $c==d$                | {102}                | {103}        |
| $e==f$                | {104}                | {105}        |
| $a==b \parallel c==d$ | {100,102}            | {103}        |
| <b>整体</b>             | <b>{100,102,104}</b> | <b>{105}</b> |

两次  $\parallel$  各回填一次左假表:  $backpatch(\{101\}, 102) \rightarrow 101$  跳 102;  $backpatch(\{103\}, 104) \rightarrow 103$  跳 104。三个真出口汇成整体 *truelist*。

## 第十次作业

- 练习6.2.3：使用图 6-43 中的翻译方案翻译下列表达式。给出每个子表达式的 **truelist** 和 **falselist**。你可以假设第一条被生成的指令的地址是 100.
- $(a==b \ \&\& \ c==d) \ \&\& \ e==f$

```

100: if a==b goto 102
101: goto _      ← 整体 F
102: if c==d goto 104
103: goto _      ← 整体 F
104: if e==f goto _ ← 整体 T
105: goto _      ← 整体 F
    
```

| 子表达式                 | truelist     | falselist            |
|----------------------|--------------|----------------------|
| $a==b$               | {100}        | {101}                |
| $c==d$               | {102}        | {103}                |
| $e==f$               | {104}        | {105}                |
| $a==b \ \&\& \ c==d$ | {102}        | {101,103}            |
| <b>整体</b>            | <b>{104}</b> | <b>{101,103,105}</b> |

两次  $\&\&$  各回填一次左真表:  $backpatch(\{100\}, 102) \rightarrow 100$  跳 102;  $backpatch(\{102\}, 104) \rightarrow 102$  跳 104。三个假出口汇成整体 **falselist**。

# 第十一次作业

截止日期: 2026.06.01

- 练习7.1.1: 考虑C 语言的函数f 和g: 按照图7-7的约定, 不考虑编译器优化, 讨论当h 调用k 而k 即将返回时运行时栈的状态, 其中h 的参数a= 2。只需要讨论返回值、参数、控制链和代码中体现的局部数据。指出

```
int k(int *, int);
int h(int a) {
    int i = a * 3 + 1;
    return k(&i, a);
}
int k(int *p, int q) {
    int s = *p + q;
    return s + 2;
}
```

1. 哪个函数在栈中为各个元素创建了所使用的空间?
2. 哪个函数写入了各个元素的值? 参数、返回值和局部变量的值是什么?
3. 这些元素属于哪个活动记录?





# 第十一次作业

截止日期：2026.06.01

... (main) 的活动记录

## h 的活动记录

|      |          |                  |
|------|----------|------------------|
| 参数 a | 2        | 调用者写入            |
| 返回值  | (待写→11)  | k 返回后由 h 写       |
| 控制链  | → main 帧 | 调用序列             |
| 局部 i | 7        | h 写 ( $=a*3+1$ ) |

## k 的活动记录 (栈顶)

|      |       |                 |
|------|-------|-----------------|
| 参数 p | &i    | 调用者 h 写         |
| 参数 q | 2     | 调用者 h 写         |
| 返回值  | 11    | k 写 ( $=s+2$ )  |
| 控制链  | → h 帧 | 调用序列            |
| 局部 s | 9     | k 写 ( $=*p+q$ ) |

h(2):  $i = 2 \times 3 + 1 = 7$ , 调用 k(&i, 2);  
k 中:  $p = \&i$ ,  $q = 2$ ,  $s = *p + q = 7 + 2 = 9$ ,  
返回  $s + 2 = 11$ 。

**关键: p 指向 h 帧中的 i。**

**p = &i**

控制链成链: k 帧 → h 帧 → main 帧, 记录“谁调用谁”;

参数 p 的值是 &i, 指向 h 帧里的局部 i——所以 k 运行期间 h 帧仍然有效;

此刻 k 即将返回: k 已写好自己的返回值 11, 而 h 的返回值尚未写入 (要等 k 返回后 h 再写);



# 第十一次作业

截止日期：2026.06.01

| 元素   | 所属活动记录 | 创建空间      | 写入者             | 值          |
|------|--------|-----------|-----------------|------------|
| 参数 a | h      | 调用者(main) | 调用者(main)       | 2          |
| 返回值  | h      | 调用者(main) | h (k 返回后; 此刻待写) | → 11       |
| 控制链  | h      | 调用序列      | 调用者(main)       | → main 帧   |
| 局部 i | h      | h         | h               | 7          |
| 参数 p | k      | 调用者 h     | 调用者 h           | &i (→ h.i) |
| 参数 q | k      | 调用者 h     | 调用者 h           | 2          |
| 返回值  | k      | 调用序列开头    | k               | 11         |
| 控制链  | k      | 调用序列      | 调用者 h           | → h 帧      |
| 局部 s | k      | k         | k               | 9          |



# 第十一次作业

截止日期: 2026.06.01

- 练习7.1.2: 考虑下面的Fibonacci函数:

嵌套在fib0中的是fib1, 它假设 $n \geq 2$ 并计算第 $n$ 个Fibonacci数; 嵌套在fib1中的是fib2, 它假设 $n \geq 4$ 。请注意, fib1和fib2都不需要检查基本情况。我们考虑从对main的调用开始, 直到(对fib0(1)的)第一次调用即将返回的时段。

1. 请描述出当时的活动记录栈, 并给出栈中的各个活动记录的访问链。
2. 假设我们使用display表来实现下图中的函数。请给出fib0(1) 的第一次调用即将返回时的display表。同时指明那时在栈中的各个活动记录中保存的display表条目。



# 第十一次作业

截止日期: 2026.06.01

- 练习7.1.2: 考虑下面的Fibonacci函数:  
嵌套在fib0中的是fib1, 它假设 $n \geq 2$ 并计算第 $n$ 个Fibonacci数;  
嵌套在fib1中的是fib2, 它假设 $n \geq 4$ 。请注意, fib1和fib2都不需要检查基本情况。我们考虑从对main的调用开始, 直到(对fib0(1)的)第一次调用即将返回的时段。
  1. 请描述出当时的活动记录栈, 并给出栈中的各个活动记录的访问链。
  2. 假设我们使用display表来实现下图中的函数。请给出fib0(1) 的第一次调用即将返回时的display表。同时指明那时在栈中的各个活动记录中保存的display表条目。



# 第十一次作业

截止日期：2026.06.01

```
fun main() {  
  let  
    fun fib0(n) =  
      let  
        fun fib1(n) =  
          let  
            fun fib2(n) = fib1(n-1) + fib1(n-2)  
          in  
            if n >= 4 then fib2(n)  
            else fib0(n-1) + fib0(n-2)  
          in  
            if n >= 2 then fib1(n) else n  
          end  
        in  
          fib0(5)  
        end;  
      }  
}
```

嵌套层次：main  $\supset$  fib0  $\supset$  fib1  $\supset$  fib2 (深度 1-4) 。



# 第十一次作业

截止日期: 2026.06.01

## 嵌套深度



外层  $\supset$  内层: fib2 能看到 fib1、fib0、main 的变量。

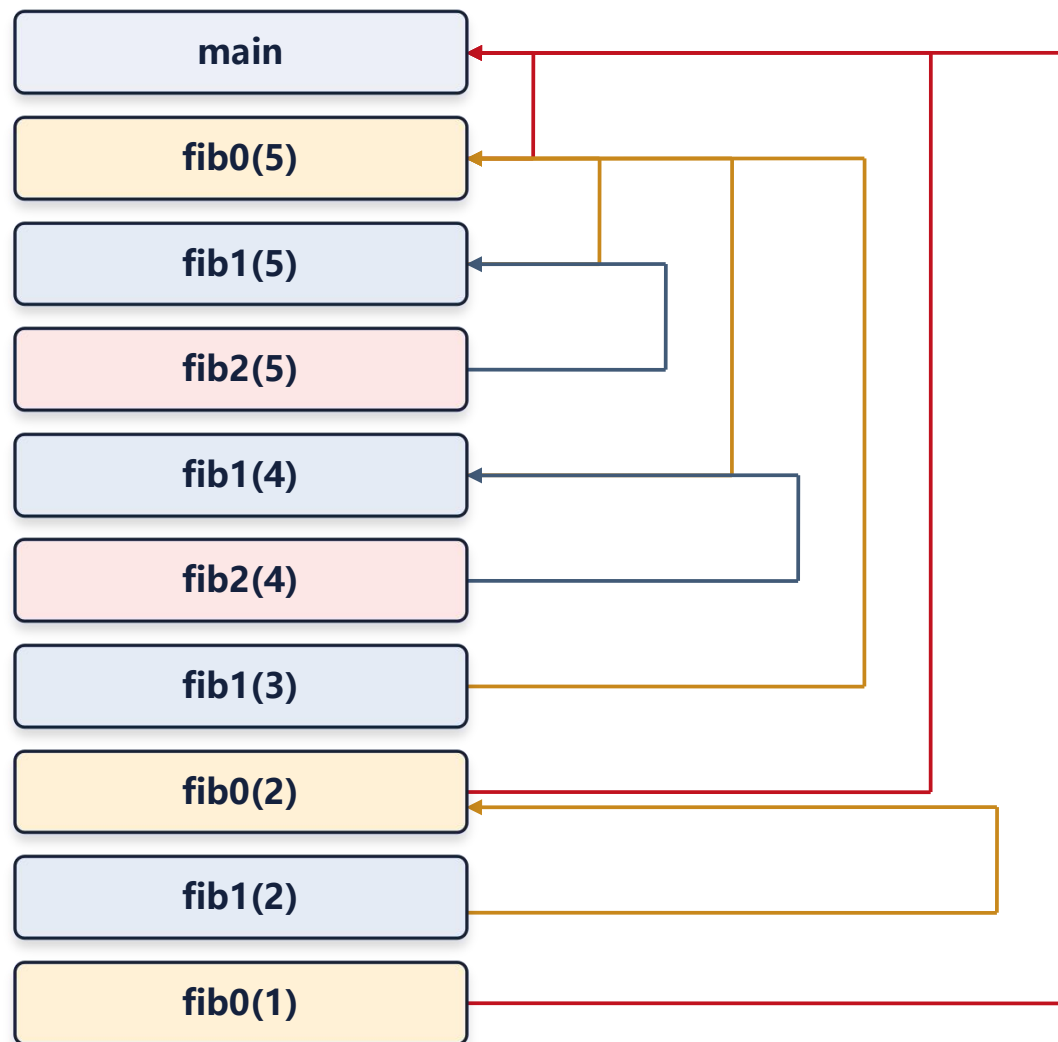
## 调用链

```
main → fib0(5)
fib0(5) n=5 ≥ 2 → fib1(5)
fib1(5) n=5 ≥ 4 → fib2(5)
fib2(5) → fib1(4) [左: fib1(n-1)]
fib1(4) n=4 ≥ 4 → fib2(4)
fib2(4) → fib1(3) [左: fib1(n-1)]
fib1(3) n=3 < 4 → fib0(2) [左: fib0(n-1)]
fib0(2) n=2 ≥ 2 → fib1(2)
fib1(2) n=2 < 4 → fib0(1) [左: fib0(n-1)]
fib0(1) n=1 < 2 → 返回 n = 1 ★
```



# 第十一次作业

截止日期：2026.06.01



## 各帧的访问链

main → 空      fib0(5) → main  
fib1(5) → fib0(5)    fib2(5) → fib1(5)  
fib1(4) → fib0(5)    fib2(4) → fib1(4)  
fib1(3) → fib0(5)    fib0(2) → main  
fib1(2) → fib0(2)    fib0(1) → main

**关键：** fib1(4) 是被 fib2(5) 调用的，  
但它的访问链指向 **fib0(5)**



# 第十一次作业

截止日期：2026.06.01

## 此刻的 d[1..4] 与各帧保存的旧条目

当前 display (fib0(1) 即将返回)

| 条目   | 指向的活动记录              |
|------|----------------------|
| d[1] | main                 |
| d[2] | <b>fib0(1) ← 当前帧</b> |
| d[3] | fib1(2) (遗留)         |
| d[4] | fib2(4) (遗留)         |

- 当前在 fib0(1) (深度 2) 中执行，只有 d[1]、d[2] 是有效的；
- d[3]、d[4] 是更深调用留下的旧值，进入深度 2 的函数时不会改动它们。

各帧进入时保存的 display 旧条目

| 活动记录           | 深度 | 保存的 d[i] 旧值           |
|----------------|----|-----------------------|
| main           | 1  | d[1] ← 空              |
| fib0(5)        | 2  | d[2] ← 空              |
| fib1(5)        | 3  | d[3] ← 空              |
| fib2(5)        | 4  | d[4] ← 空              |
| fib1(4)        | 3  | d[3] ← fib1(5)        |
| fib2(4)        | 4  | d[4] ← fib2(5)        |
| fib1(3)        | 3  | d[3] ← fib1(4)        |
| <b>fib0(2)</b> | 2  | <b>d[2] ← fib0(5)</b> |
| fib1(2)        | 3  | d[3] ← fib1(3)        |
| <b>fib0(1)</b> | 2  | <b>d[2] ← fib0(2)</b> |